

Data at Scale & Distributed Systems

The end of Part III (scaling the data tier) and the start of Part IV (the theory).

- 15 NoSQL Databases** — key-value, document, wide-column, graph
- 16 Replication** — copies for availability and read scale
- 17 Partitioning & Sharding** — splitting data across machines
- 18 Consistent Hashing** — adding nodes without reshuffling everything
- 19 Fallacies & Hard Truths of Distributed Systems** — opens Part IV
- 20 CAP Theorem & PACELC** — the consistency vs availability choice

Chapters 15–18 finish scaling the data tier; 19–20 begin the theory of why distributed systems are hard.

CHAPTER 15

NoSQL Databases

Part III · Data & Storage — trading relational features for scale and flexibility.

LEARNING OBJECTIVES

- Explain what NoSQL is and what it trades away versus a relational database.
- Describe the four families — key-value, document, wide-column, graph — and their uses.
- Explain access-pattern-first modeling.
- Decide between SQL and NoSQL for a given workload.

REAL-WORLD ANALOGY — SPECIALIZED STORAGE ROOMS

A relational database is one well-organized general store. NoSQL is a set of **specialized rooms**: a coat-check that trades a ticket for a coat (key-value), folders holding whole documents (document), a vast warehouse of flexible shelving (wide-column), and a map of who-knows-whom (graph). Each is superb at one shape of data and clumsy at others.

15.1 The problem

Relational databases are the default (Chapter 11), but they have two pain points at very large scale: a **rigid schema** and difficulty scaling **horizontally** (JOINS and ACID across many machines are hard, Chapter 14). **NoSQL** (“Not Only SQL”) is a family of databases that relax relational guarantees to gain **flexible schemas, easy horizontal scaling, and very high throughput** — typically giving up JOINS, some ACID (often settling for eventual consistency), and rich ad-hoc queries.

15.2 The four families

Key-Value

"user:42" → { name, email }
fastest lookups — caching, sessions

Document

{ id, name, posts: [...] }
flexible, nested — profiles, catalogs

Wide-Column

row → colA:... colB:... colC:...
huge write throughput — time-series

Graph

 relationships first — social, fraud

Family	Shape & examples	Best for
Key-Value	Key → opaque value. Redis, DynamoDB.	Caching, sessions, simple fast lookups.
Document	JSON-like documents. MongoDB, Couchbase.	Flexible/nested data: profiles, content, catalogs.
Wide-Column	Rows with dynamic columns. Cassandra, HBase, Bigtable.	Massive write volume, time-series, huge scale.

Graph	Nodes + edges. Neo4j.	Highly-connected data: social graphs, recommendations.
--------------	-----------------------	--

15.3 Model around your access patterns

KEY INSIGHT — QUERY-FIRST, NOT STRUCTURE-FIRST

In relational design you normalize, then query with JOINS. In NoSQL there are usually **no JOINS**, so you model the data *around the queries you'll run* — embedding and denormalizing so each read hits one place. Design the reads first, then shape the data to serve them.

15.4 SQL vs NoSQL

Choose SQL when...	Choose NoSQL when...
Data is structured and related; you need JOINS, ad-hoc queries, and strong ACID.	You need massive horizontal scale, a flexible schema, simple access patterns, or a specific shape (e.g. a graph).

Many systems use **both** — “polyglot persistence”: a relational database for core transactional data, Redis for caching, a document store for content, a graph store for the social graph.

Common mistakes

WATCH OUT

- Reaching for NoSQL by default when a relational database fits (Chapter 11).
- Modeling NoSQL like a relational schema (normalized, JOIN-dependent) instead of around access patterns.
- Assuming all NoSQL is “eventually consistent” or lacks transactions — capabilities vary widely.
- Picking a family that doesn't match the data shape (e.g. a KV store for deeply connected data).

Interview questions

1. What does NoSQL trade away versus relational, and what does it gain?
2. Describe the four NoSQL families and a use case for each.
3. Why do you model NoSQL around access patterns?
4. When would you choose SQL over NoSQL, and vice versa?
5. What is polyglot persistence?

Hands-on exercises

1. For each: pick a NoSQL family — a session store, a product catalog, a friend graph, IoT sensor readings.
2. Model a user profile with embedded recent posts as a single document.

3. Explain why a graph store answers “friends of friends” better than a relational database.
4. List two capabilities you lose by moving from relational to a key-value store.

Mini project

SAME DATA, TWO SHAPES

Take a small blog domain and model it twice: once as normalized relational tables, and once as a document store where each post embeds its author summary and recent comments. Write the query “show a post with author and comments” against both and compare the effort — a JOIN versus a single document fetch. Note what each makes easy and what each makes hard.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — PICK YOUR STORES

Decide a **polyglot** storage plan: relational for core user/post data and anything transactional, a key-value cache (from Chapter 8), and consider a **graph** store for the follow graph and a **wide-column** store for the high-volume feed/activity stream. Justify each choice by data shape and access pattern from your Chapter 5 numbers.

Chapter summary

- **NoSQL** relaxes relational guarantees (schema, JOINs, some ACID) to gain flexible schemas, horizontal scale, and throughput.
- Four families: **key-value** (fast lookups), **document** (flexible/nested), **wide-column** (massive writes), **graph** (connections).
- Model NoSQL **around access patterns**, embedding and denormalizing.
- SQL is the default; choose NoSQL for scale, flexibility, or a specific shape — and use **both** where it fits.

CHAPTER 16

Replication

Part III · Data & Storage — copies of data for availability and read scale.

LEARNING OBJECTIVES

- Explain why we replicate data and what it buys.
- Describe leader-follower replication and the read/write split.
- Compare synchronous and asynchronous replication.
- Explain replication lag and the anomalies it causes.
- Explain failover and the split-brain risk.

REAL-WORLD ANALOGY — BRANCH-OFFICE COPIES OF THE LEDGER

A company keeps its master ledger at headquarters and mirrors it to branch offices. Branches answer local questions quickly (read scale), and if headquarters burns down a branch copy survives (availability). But a branch copy can be slightly out of date — that lag is the whole subtlety of replication.

16.1 The problem

One database is a single point of failure and a read bottleneck (Chapters 6, 10). **Replication** keeps copies of the data on multiple machines, buying **high availability** (survive a node loss), **read scaling** (spread reads across copies), **durability** (extra copies), and **geo-locality** (a copy near users).

16.2 Leader-follower replication

The most common scheme: one **leader** (primary) accepts all writes and streams its changes to one or more **followers** (replicas). Reads can be served by any copy, so read-heavy workloads scale by adding followers.



16.3 Synchronous vs asynchronous

Synchronous

Asynchronous

Leader waits for a follower to confirm before acknowledging the write. Safe (no data loss), but slow, and blocks if a follower is down.

Leader acknowledges immediately and replicates in the background. Fast, but recent writes can be **lost** if the leader dies, and followers show **lag**.

Many systems use **semi-synchronous**: wait for one follower (safety) but not all (speed).

16.4 Replication lag and its anomalies

READING FROM A STALE FOLLOWER

Async followers trail the leader by a **lag**, so reading from one can return stale data — causing the **read-your-own-writes** problem (you post a comment, then don't see it because your read hit a lagging follower). Mitigations: read your own recent writes from the leader, or ensure a user's reads are **monotonic** (never go backward in time). This is **eventual consistency** in action.

16.5 Multi-leader and leaderless (brief)

Multi-leader: several leaders accept writes (e.g. one per region) — great for multi-region latency, but writes can **conflict** and must be resolved. **Leaderless** (Dynamo-style): any replica takes writes, and reads/writes use **quorums** to stay consistent (built on consistent hashing, Chapter 18, and quorum math, Part IV).

16.6 Failover and split-brain

PROMOTING A NEW LEADER IS DANGEROUS

When the leader dies, a follower is **promoted**. This must handle: detecting the failure (dead vs just slow?), choosing the most up-to-date follower, and avoiding **split-brain** — two nodes both believing they're leader and accepting conflicting writes. Async replication also means the old leader's un-replicated writes can be **lost**.

Common mistakes

WATCH OUT

- Assuming followers are perfectly up to date — ignoring replication lag.
- Serving a user's own just-written data from a lagging follower.
- Believing replication scales **writes** — it doesn't (every copy takes every write); that's sharding's job (Chapter 17).
- Automatic failover without split-brain protection.

Interview questions

1. Why replicate? What does it buy and what doesn't it solve?
2. Explain leader-follower replication and the read/write split.
3. Compare synchronous and asynchronous replication.
4. What is replication lag, and what problems does it cause?

5. What is split-brain, and why is failover tricky?

Hands-on exercises

1. Diagram a leader with three followers and label where reads and writes go.
2. Give a user-facing bug caused by replication lag and a fix.
3. Decide sync vs async for: a bank ledger; a social feed. Justify.
4. Explain why adding followers doesn't increase write capacity.

Mini project

SET UP A REPLICA AND OBSERVE LAG

Configure a database with one leader and one follower. Write data to the leader and read it from the follower; introduce load and observe **replication lag**. Then reproduce the read-your-own-writes anomaly and fix it by routing recent reads to the leader.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE REPLICATION PLAN

Add **read replicas** to your database to serve your read-heavy feed traffic. Decide sync vs async, how you'll handle read-your-own-writes for a user's own posts, and your failover approach. Note that replication handles reads and availability — but not write volume or data size, which is why Chapter 17 (sharding) comes next.

Chapter summary

- **Replication** keeps copies on multiple machines for availability, read scaling, durability, and geo-locality.
- **Leader-follower** is the common model: writes to the leader, reads from followers.
- **Sync** is safe but slow; **async** is fast but risks lost writes and shows **lag**.
- Lag causes stale/read-your-own-writes anomalies — eventual consistency.
- Replication does **not** scale writes or storage, and failover risks **split-brain**.

CHAPTER 17

Partitioning & Sharding

Part III · Data & Storage — splitting data across machines to scale writes and size.

LEARNING OBJECTIVES

- Explain why replication alone can't scale writes or storage.
- Compare range, hash, and directory partitioning.
- Choose a good shard key and avoid hotspots.
- Explain the challenges: cross-shard queries, transactions, and rebalancing.

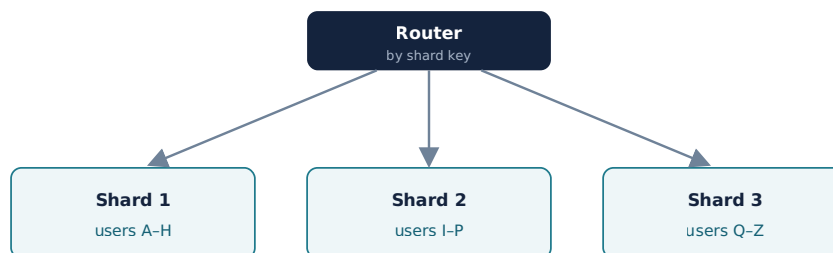
REAL-WORLD ANALOGY — SPLITTING THE PHONE BOOK

A single phone book for a whole country would be impossibly thick. So you split it into volumes — A-H, I-P, Q-Z — each a manageable book. To find someone you go straight to the right volume. **Sharding** splits your data the same way, each machine holding one slice.

17.1 The problem

Replication (Chapter 16) scales reads and gives availability, but **every copy still holds all the data and takes every write**. So it can't help when the data is too big for one machine, or the write rate is too high. **Partitioning** (a.k.a. **sharding**) splits the data into pieces spread across machines, so each holds only a subset — scaling both storage and write throughput.

17.2 Partitioning strategies



Strategy	How	Trade-off
Range	Split by key ranges (A-H, dates).	Great range queries; risks hotspots (uneven ranges).
Hash	<code>hash(key) % N</code> chooses the shard.	Even spread; loses efficient range queries.
Directory	A lookup table maps key → shard.	Flexible; the lookup is an extra hop and a dependency.

17.3 Choosing the shard key

KEY INSIGHT — THE SHARD KEY DECIDES EVERYTHING

A good shard key has **high cardinality**, spreads load **evenly**, and matches your **access patterns** (so common queries hit one shard). A bad key creates **hotspots** — the “celebrity problem,” where one hugely popular key (a star’s account) overwhelms a single shard. Choose the key for how data is written and read, not just what’s convenient.

17.4 The challenges

- **Cross-shard queries:** a query needing data from many shards must fan out and merge — slow. Design to keep common queries on one shard.
- **Cross-shard transactions:** ACID across shards is hard and slow (Part IV) — avoid where possible.
- **Rebalancing:** adding a shard shouldn’t reshuffle everything — the naive $\text{hash} \% N$ does exactly that, which is why **consistent hashing** (Chapter 18) exists.
- **Hotspots:** handle celebrity keys with extra replication or by splitting them.

17.5 Sharding + replication together

The standard production setup combines both: **shard** the data for scale, and **replicate each shard** for availability. Each shard is a small leader-follower group. You get horizontal scale *and* fault tolerance.

Common mistakes

WATCH OUT

- Sharding before you need it — it adds major complexity (scale up and replicate first).
- A shard key that causes hotspots or forces most queries to hit all shards.
- Using $\text{hash} \% N$ and facing a full reshuffle when you add a node.
- Relying on cross-shard JOINS and transactions.

Interview questions

1. Why can’t replication scale writes or storage — and how does sharding?
2. Compare range, hash, and directory partitioning.
3. What makes a good shard key? What is the celebrity/hotspot problem?
4. Why are cross-shard queries and transactions hard?
5. How do sharding and replication combine?

Hands-on exercises

1. Pick a shard key for a messaging app and explain how it spreads load.
2. Show why $\text{hash}(\text{key}) \% 4 \rightarrow \% 5$ remaps most keys.
3. Identify a query that would need to hit all shards, and redesign to avoid it.

4. Sketch three shards, each replicated with one follower.

Mini project

SHARD A DATASET BY HAND

Take a dataset of a few thousand records and split it across three “shards” (three tables or files) by a chosen shard key. Route lookups to the right shard. Then measure how a query by shard key (one shard) compares to a query that must scan all shards — feeling the cost of cross-shard access.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE SHARDING PLAN

Decide when your data outgrows a single (replicated) database. Choose a **shard key** (e.g. `user_id`) and a strategy, and check it against your hottest queries and the celebrity problem. Identify which operations would become cross-shard, and how you’ll combine sharding with the replicas from Chapter 16. Note that rebalancing motivates Chapter 18.

Chapter summary

- **Sharding** splits data across machines to scale **writes and storage** — what replication can’t do.
- Strategies: **range** (range queries, hotspot risk), **hash** (even, no ranges), **directory** (flexible, extra hop).
- The **shard key** is the critical choice — high cardinality, even, access-pattern-aligned; beware hotspots.
- Cross-shard queries/transactions are hard; rebalancing motivates consistent hashing.
- Production combines **sharding + replication**.

CHAPTER 18

Consistent Hashing

Part III · Data & Storage — adding and removing nodes without reshuffling everything.

LEARNING OBJECTIVES

- Explain why $\text{hash}(\text{key}) \% N$ breaks when the number of nodes changes.
- Explain the hash ring and how a key finds its node.
- Explain why adding/removing a node moves only $\sim 1/N$ of keys.
- Explain virtual nodes and where consistent hashing is used.

REAL-WORLD ANALOGY — A ROUND TABLE OF GUESTS

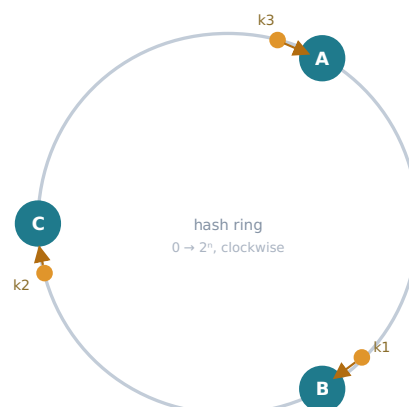
Seat guests around a circular table. Each dish is served by the next guest clockwise. If one guest leaves or a new one sits down, only the dishes *near that seat* change hands — everyone else keeps their assignments. Consistent hashing arranges data and servers on a circle so a change disturbs only a small neighborhood.

18.1 The problem with $\text{hash} \% N$

A simple way to spread keys over N nodes is $\text{node} = \text{hash}(\text{key}) \% N$. It distributes evenly — until N changes. Add or remove one node and **N changes, so almost every key maps to a different node**. For a cache that means nearly every entry suddenly misses; for a shard set it means moving nearly all the data. We need a scheme where a node change moves only a **small fraction** of keys.

18.2 The hash ring

Hash both **keys and nodes** onto the same circular space (say 0 to 2^n). A key is owned by the **first node clockwise** from the key's position on the ring.



Each key (k_1, k_2, k_3) is stored on the **next node clockwise**: $k_1 \rightarrow B$, $k_2 \rightarrow C$, $k_3 \rightarrow A$.

18.3 Adding or removing a node

KEY INSIGHT — ONLY $\sim 1/N$ OF KEYS MOVE

When you add a node, it lands somewhere on the ring and takes over only the keys between it and its **predecessor** — roughly $1/N$ of all keys. Every other key keeps its owner. Removing a node hands its keys to the **next node clockwise**, and again nothing else moves. Compared with $\% N$ reshuffling almost everything, this is the entire point.

18.4 Virtual nodes

With only a few nodes, the ring is uneven (one node may own a big arc), and losing a node dumps *all* its load onto a single neighbor. The fix: give each physical node **many virtual points** scattered around the ring. Load is spread smoothly, and when a node fails its keys fan out across **many** nodes instead of one. Virtual nodes make consistent hashing practical.

18.5 Where it's used

Distributed caches (Memcached client libraries), **Dynamo/Cassandra** partitioning, some load balancers, and CDNs all use consistent hashing so that scaling the cluster doesn't trigger a mass data movement or a cache-wide miss storm.

Common mistakes

WATCH OUT

- Using `hash % N` for a cache or shard set that will grow — guaranteeing a reshuffle later.
- Consistent hashing **without virtual nodes**, leading to uneven load and lumpy failover.
- Forgetting that a removed node's load lands entirely on its clockwise neighbor (without vnodes).

Interview questions

1. Why does `hash(key) % N` break when N changes?
2. How does the hash ring assign keys to nodes?
3. Why does adding a node move only $\sim 1/N$ of keys?
4. What problem do virtual nodes solve?
5. Name systems that use consistent hashing and why.

Hands-on exercises

1. With 4 nodes, compute how many of 100 keys remap under `% 4` $\rightarrow$ `% 5`.
2. Draw a ring with three nodes and place four keys; name each key's owner.
3. Add a fourth node to your ring and mark which keys move.
4. Explain how virtual nodes change what happens when a node fails.

Mini project

IMPLEMENT A TINY CONSISTENT-HASH RING

Write a small consistent-hash ring: hash nodes and keys onto a circle, and look up each key's owner (next node clockwise). Distribute 10,000 keys over 4 nodes and print the per-node counts. Add a 5th node and measure the fraction of keys that move (should be $\sim 1/5$). Then add virtual nodes and watch the distribution even out.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — SCALABLE PARTITIONING

Specify how your sharded data (Chapter 17) is placed with **consistent hashing** so you can add capacity without a mass reshuffle. Decide a virtual-node count for even load, and describe what happens to keys and cache entries when you add or lose a node during peak traffic.

Chapter summary

- `hash % N` remaps almost all keys when N changes — unacceptable for caches and shards.
- **Consistent hashing** places keys and nodes on a ring; a key goes to the next node clockwise.
- Adding/removing a node moves only $\sim 1/N$ of keys.
- **Virtual nodes** give even load and graceful failover.
- Used in Dynamo/Cassandra, distributed caches, load balancers, and CDNs.

CHAPTER 19

Fallacies & Hard Truths of Distributed Systems

Part IV · Distributed Systems Theory — why the network changes everything.

LEARNING OBJECTIVES

- State the eight fallacies of distributed computing and why each is false.
- Explain the core hard truths: partial failure, unreliable networks, no global clock.
- Explain why you can't tell a dead node from a slow one.
- Translate these truths into design principles.

REAL-WORLD ANALOGY — ASSUMPTIONS THAT ONLY HOLD IN ONE ROOM

Coordinating people in a single room is easy: you can see everyone, hear instantly, and know who's present. Coordinate people across different cities by unreliable mail and the "obvious" assumptions collapse — messages get lost, arrive late or out of order, and you can't tell if someone is ignoring you or simply hasn't received your letter. Distributed systems live in the second world.

19.1 Part IV begins

Part III spread data across machines (replication, sharding). The moment your system spans multiple machines connected by a network, a new class of problems appears. Part IV is the **theory** of why distributed systems are hard — starting with the classic fallacies and the hard truths beneath them.

19.2 The eight fallacies of distributed computing

Fallacy (people assume...)	Reality & consequence
1. The network is reliable	It drops and delays messages → use timeouts, retries, idempotency (Chapter 3).
2. Latency is zero	Every hop costs time → minimize round trips, batch, cache.
3. Bandwidth is infinite	Payload size and egress cost matter → compress, page, trim.
4. The network is secure	It isn't → encrypt in transit, authenticate, assume zero-trust.
5. Topology doesn't change	Nodes come and go → use service discovery, don't hard-code hosts.
6. There is one administrator	Many parties, configs, versions → expect coordination and mismatch.
7. Transport cost is zero	Serialization and bandwidth cost real money/CPU → budget for it.
8. The network is homogeneous	Mixed hardware/protocols → design for interoperability.

19.3 The hard truths beneath them

- **Partial failure:** unique to distributed systems — some parts work while others fail, and you often can't tell which. A single machine is all-or-nothing; a distributed system is not.
- **Unreliable networks:** messages can be lost, duplicated, delayed, or reordered.
- **No global clock:** machines' clocks disagree, so "what happened first" is genuinely ambiguous (Chapter 23 covers logical clocks).
- **Dead vs slow is undecidable:** if a node doesn't answer, you cannot know whether it crashed or is merely slow (or the network dropped the reply). All you have is a **timeout** — a guess.

KEY INSIGHT — THIS AMBIGUITY DRIVES EVERYTHING

Because you can never be *certain* a node is dead, every distributed algorithm — failover, leader election, consensus — is really about making safe decisions under uncertainty. Much of Part IV exists to cope with this single, stubborn fact.

19.4 Design implications

- **Design for failure:** assume every remote call can fail; test it deliberately (chaos engineering).
- **Timeouts everywhere**, plus retries with backoff and **idempotency** so retries are safe.
- **Embrace eventual consistency** where strict consistency is too costly (Chapters 20–21).
- **Observability:** you can't debug what you can't see — logs, metrics, tracing (Chapter 50).

Common mistakes

WATCH OUT

- Writing remote calls as if they were local (no timeout, no retry, no failure path).
- Assuming clocks agree, or that message order is preserved.
- Treating a slow node as definitely dead (or vice versa) and acting rashly.
- Ignoring partial failure — assuming operations are all-or-nothing across machines.

Interview questions

1. List several fallacies of distributed computing and their consequences.
2. What is partial failure, and why is it unique to distributed systems?
3. Why can't you distinguish a dead node from a slow one?
4. Why is "no global clock" a problem?
5. How do these truths shape real designs?

Hands-on exercises

1. For each fallacy, name one real bug it causes and a mitigation.
2. Describe a scenario where a timeout fires but the node was actually alive.

3. Explain why retrying a non-idempotent call after a timeout is dangerous.
4. List three things you'd add to make a remote call robust.

Mini project

MAKE AN UNRELIABLE CALL SURVIVABLE

Write a client that calls a flaky service (make it randomly delay, drop, or error). Add a timeout, retries with exponential backoff, and an idempotency key so retries don't double-apply. Then inject a "slow but alive" case and show how your timeout can misfire — and what you do about it.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — A FAILURE INVENTORY

List every network boundary in your architecture (client→gateway, app→cache, app→DB, service→service). For each, note what happens on timeout, whether the call is idempotent, and how you detect and recover from partial failure. This inventory is the foundation for the reliability patterns in Part VI.

Chapter summary

- The **eight fallacies** are false assumptions (reliable network, zero latency, infinite bandwidth, secure, static topology, one admin, zero transport cost, homogeneous).
- Beneath them: **partial failure**, unreliable networks, **no global clock**, and the impossibility of telling **dead from slow**.
- These force designing **for failure**: timeouts, retries, idempotency, eventual consistency, and observability.

CHAPTER 20

CAP Theorem & PACELC

Part IV · Distributed Systems Theory — the consistency vs availability choice.

LEARNING OBJECTIVES

- State the CAP theorem and why P is not optional.
- Explain the CP vs AP choice with real examples.
- Correct the common misreading of CAP.
- Explain how PACELC extends CAP to normal operation.

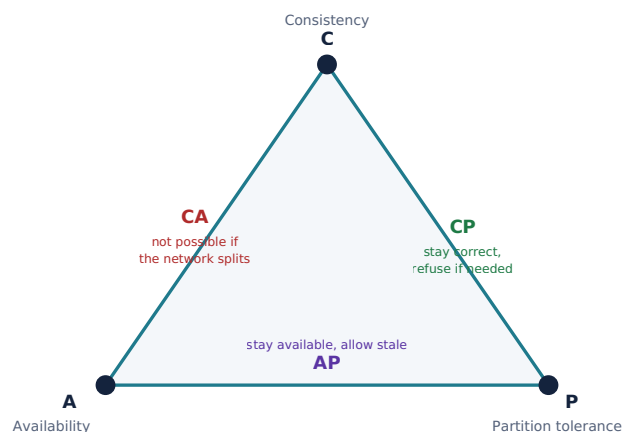
REAL-WORLD ANALOGY — TWO CLERKS CUT OFF FROM EACH OTHER

Two ticket clerks share one seating chart but their phone line drops (a network partition). A customer asks each to sell the last seat. They can either **refuse to sell** until the line is back (staying *consistent* but unavailable) or **both sell it** and sort out the clash later (staying *available* but inconsistent). During the outage they cannot have both. That is CAP.

20.1 The problem

In any replicated or sharded system, network **partitions** will happen (Chapter 19). The **CAP theorem** says that during a partition a distributed system can guarantee at most two of: **Consistency** (every read sees the latest write), **Availability** (every request gets a non-error response), and **Partition tolerance** (it keeps working despite dropped messages).

20.2 Really a choice of two



Because networks **do** partition, **P is mandatory** in any real distributed system. So the real choice, *when a partition happens*, is **CP vs AP**:

CP — choose consistency

AP — choose availability

Refuse or error rather than serve stale/conflicting data. Banks, inventory, anything that must be correct.

Keep responding, allow temporary staleness, reconcile later (eventual consistency). Feeds, shopping carts, likes.

20.3 The common misreading

KEY INSIGHT — CAP IS ONLY ABOUT THE PARTITION MOMENT

CAP does **not** mean “pick two forever.” When the network is healthy, a good system provides *both* consistency and availability. The trade-off is forced **only during a partition**. “CA” systems (consistency + availability, no partition tolerance) don’t meaningfully exist across a network, because you can’t opt out of partitions.

20.4 PACELC — the everyday trade-off

CAP only speaks about partitions, but there’s a trade-off even in normal times. **PACELC**: *if Partition, choose **A** or **C**; Else (normal operation), choose **L**atency or **C**onsistency*. Waiting for all replicas to agree is more consistent but slower; answering from the nearest replica is faster but may be stale. PACELC captures the latency-vs-consistency dial you face *all the time*, not just during failures.

20.5 Choosing for your system

Match the choice to the domain: money and inventory lean **CP / consistency**; feeds, carts, and social signals lean **AP / latency**. Many databases make this **tunable** per operation (e.g. quorum settings), so you can pick strong or fast on a query-by-query basis (Chapter 21 goes deeper on consistency models).

Common mistakes

WATCH OUT

- Reading CAP as “pick two, always” instead of “during a partition, pick C or A.”
- Believing you can build a “CA” distributed system — you can’t skip partition tolerance.
- Forcing strong consistency on data that would be fine eventually consistent (hurting availability/latency needlessly).
- Ignoring the Else half of PACELC — the latency/consistency trade during normal operation.

Interview questions

1. State the CAP theorem. Why is P not optional?
2. Give a CP example and an AP example, and justify each.
3. What’s the common misunderstanding of CAP?
4. What does PACELC add?
5. How would you choose consistency vs availability for a shopping cart vs a bank balance?

Hands-on exercises

1. Classify as CP or AP: a payment ledger, a like counter, a seat-booking system, a DNS cache.
2. Describe what a CP and an AP system each do during a partition for the same request.
3. Write the PACELC profile (e.g. PA/EL vs PC/EC) for a system you know.
4. Give one operation you'd run at strong consistency and one at eventual, in the same app.

Mini project

FEEL THE TRADE-OFF

Take a replicated key-value store that lets you set read/write consistency per request. Perform the same write-then-read at strong consistency and at eventual consistency, under simulated replica lag, and observe when the eventual read returns stale data. Then reason about which of your app's operations belong in each mode.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — CP/AP PER FEATURE

Go feature by feature and assign a stance: the follow/like counters and feed can be **AP** (eventually consistent); anything involving payments, account settings, or uniqueness (usernames) should be **CP**. Write the PACELC profile for your core datastore and note where you'll expose tunable consistency. This decision drives the consistency-model details in Chapter 21.

Chapter summary

- **CAP:** during a network partition, guarantee at most two of Consistency, Availability, Partition tolerance.
- Partitions are unavoidable, so **P is mandatory** — the real choice is **CP vs AP**.
- CAP's trade-off applies **only during a partition**; healthy systems offer both.
- **PACELC** adds the normal-operation dial: Else, choose **Latency or Consistency**.
- Choose per domain and per operation; many stores make it **tunable**.

COMING NEXT

Chapter 21 — Consistency Models makes “consistency” precise — strong, eventual, causal, read-your-writes — and shows how systems offer and combine them.